Manipulation des Bases de Données en C# avec ADO.NET et Entity Framework

Par Robert DIASSÉ

Dans ce cours, nous allons explorer **toutes les fonctionnalités essentielles** pour manipuler une base de données en C# en utilisant deux approches principales :

1. **ADO.NET** (approche bas niveau, accès direct à la base)
2. **Entity Framework (EF Core)** (approche orientée objet, plus moderne et efficace)

Nous allons également couvrir :

- **Les types de bases de données compatibles avec C#**
- **La connexion à une base de données**
- **Les opérations CRUD (Create, Read, Update, Delete)**
- **Les requêtes avancées et jointures**
- **Les transactions et gestion des erreurs**
- **L'ORM Entity Framework Core**
- **Les migrations et manipulation avancée des données**

---

1. Prérequis

Avant de commencer, il faut :

- Avoir installé **Visual Studio**
- Avoir une base de données SQL Server (ou MySQL/PostgreSQL selon votre choix)
- Connaître les bases du SQL (SELECT, INSERT, UPDATE, DELETE)

---

2. Connexion à une Base de Données avec ADO.NET

Installation de SQL Server et Création d'une Base de Données

1. Installer **SQL Server** et **SQL Server Management Studio (SSMS)**
2. Créer une base de données nommée **"GestionEcole"**
3. Créer une table **"Etudiants"**

Script SQL de Création de la Table

```sql
CREATE DATABASE GestionEcole;
USE GestionEcole;

CREATE TABLE Etudiants (
    Id INT PRIMARY KEY IDENTITY,
    Nom VARCHAR(50),
    Prenom VARCHAR(50),
    Age INT
);
```

Connexion à la Base avec ADO.NET

ADO.NET permet d'interagir avec la base via `SqlConnection`.

Exemple de Connexion

```csharp
using System;
using System.Data.SqlClient;

class Program
{
    static void Main()
    {
        string connectionString =
"Server=localhost;Database=GestionEcole;Trusted_Connection=True;";

        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            try
            {
                connection.Open();
                Console.WriteLine("Connexion réussie à la base de données !");
            }
            catch (Exception ex)
            {
                Console.WriteLine("Erreur de connexion : " + ex.Message);
            }
        }
    }
}
```

---

3. Opérations CRUD avec ADO.NET

Insertion d'un Enregistrement (Create)

```
using System;
using System.Data.SqlClient;

class Program
{
    static void Main()
    {
        string connectionString =
"Server=localhost;Database=GestionEcole;Trusted_Connection=True;";

        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            connection.Open();

            string query = "INSERT INTO Etudiants (Nom, Prenom, Age) VALUES (@nom,
@prenom, @age)";
            using (SqlCommand command = new SqlCommand(query, connection))
            {
                command.Parameters.AddWithValue("@nom", "Dupont");
                command.Parameters.AddWithValue("@prenom", "Jean");
                command.Parameters.AddWithValue("@age", 22);

                int rowsAffected = command.ExecuteNonQuery();
                Console.WriteLine($"{rowsAffected} ligne(s) insérée(s).");
            }
        }
    }
}
```

Lecture des Données (Read)

```
using System;
using System.Data.SqlClient;

class Program
{
    static void Main()
    {
        string connectionString =
"Server=localhost;Database=GestionEcole;Trusted_Connection=True;";

        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            connection.Open();

            string query = "SELECT * FROM Etudiants";
            using (SqlCommand command = new SqlCommand(query, connection))
            {
                using (SqlDataReader reader = command.ExecuteReader())
```

```
                {
                    while (reader.Read())
                    {
                        Console.WriteLine($"ID: {reader["Id"]}, Nom:
{reader["Nom"]}, Prénom: {reader["Prenom"]}, Age: {reader["Age"]}");
                    }
                }
            }
        }
    }
}
```

Mise à Jour d'un Enregistrement (Update)

```
string query = "UPDATE Etudiants SET Age = @age WHERE Id = @id";
using (SqlCommand command = new SqlCommand(query, connection))
{
    command.Parameters.AddWithValue("@age", 25);
    command.Parameters.AddWithValue("@id", 1);

    int rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} ligne(s) mise(s) à jour.");
}
```

Suppression d'un Enregistrement (Delete)

```
string query = "DELETE FROM Etudiants WHERE Id = @id";
using (SqlCommand command = new SqlCommand(query, connection))
{
    command.Parameters.AddWithValue("@id", 1);

    int rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} ligne(s) supprimée(s).");
}
```

---

4. Manipulation de Base de Données avec Entity Framework Core (EF Core)

Installation d'Entity Framework Core

Ajoutez Entity Framework Core à votre projet via NuGet :

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Configuration du DbContext

```
using Microsoft.EntityFrameworkCore;

public class ApplicationDbContext : DbContext
{
    public DbSet<Etudiant> Etudiants { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {

optionsBuilder.UseSqlServer("Server=localhost;Database=GestionEcole;Trusted_Connec
tion=True;");
    }
}

public class Etudiant
{
    public int Id { get; set; }
    public string Nom { get; set; }
    public string Prenom { get; set; }
    public int Age { get; set; }
}
```

Ajout d'un Etudiant

```
using System;

class Program
{
    static void Main()
    {
        using (var context = new ApplicationDbContext())
        {
            var etudiant = new Etudiant { Nom = "Doe", Prenom = "John", Age = 23
};
            context.Etudiants.Add(etudiant);
            context.SaveChanges();

            Console.WriteLine("Étudiant ajouté !");
        }
    }
}
```

Requête Avancée avec EF Core

Filtrer et trier les étudiants

```
using System.Linq;

class Program
{
    static void Main()
    {
        using (var context = new ApplicationDbContext())
        {
            var etudiants = context.Etudiants
                .Where(e => e.Age > 20)
                .OrderBy(e => e.Nom)
                .ToList();

            foreach (var e in etudiants)
            {
                Console.WriteLine($"{e.Nom} {e.Prenom} - {e.Age} ans");
            }
        }
    }
}
```

---

5. Exercices Pratiques

1. **Créer une table Professeurs et implémenter CRUD avec ADO.NET**
2. **Utiliser Entity Framework pour gérer une table Cours (ID, Nom, Durée)**
3. **Créer une jointure entre Étudiants et Cours (relation Many-to-Many)**

---

6. Conclusion et Perspectives

- **ADO.NET** est idéal pour un contrôle fin des requêtes SQL.
- **Entity Framework** simplifie la gestion des bases avec une approche orientée objet.
- **Next Step** : Gestion des APIs avec **ASP.NET Core et bases de données** !